

Rapport de Test & Guide d'Enregistrement — Classical Ciphers

Objectif : Tester EXCLUSIVEMENT les chiffrements classiques (Classical Ciphers) devant la caméra.

■ ■ Durée estimée : ~10-15 minutes.

■ ■ Avant de commencer

Ouvre un terminal dans le dossier crypto/ et garde-le à l'écran.

```
cd C:\Users\HP\OneDrive\Bureau\crypto
```

PARTIE 1 — Chiffrements Classiques (Classical_Ciphers/)

1.1 — César (Cesar.py)

```
python Classical_Ciphers/Cesar.py
```

Étape	Action	Entrées à taper	Résultat attendu
①	Chiffrer	Option 1 → Clé 3 → Mode u → cryptographie	fubswrjudsklh
②	Déchiffrer	Option 2 → Clé 3 → Mode u → fubswrjudsklh	cryptographie
③	Force brute	Option 3 → Mode u → fubswrjudsklh	26 clés affichées, ■ clé 3
④	Indice coïncidence	Option 4 → Mode t	IC ≈ 0.074, clé trouvée
⑤	Quitter	Option 5	—

■ César décale chaque lettre de k positions. Ici k=3 → 'c' devient 'f'. La force brute teste les 26 clés.

1.2 — Affine (affine.py)

```
python Classical_Ciphers/affine.py
```

Étape	Action	Entrées à taper	Résultat attendu
①	Chiffrer	Option 1 → securite → a=5 → b=8	ucsepwzc
②	Déchiffrer	Option 2 → a=5 → b=8 → ucsepwzc	securite
③	Quitter	q	—

■ Affine : $y = (a \cdot x + b) \bmod 26$. a doit être premier avec 26.

1.3 — Vigenère (vigenere.py)

```
python Classical_Ciphers/vigenere.py
```

Étape	Action	Entrées à taper	Résultat attendu
①	Chiffrer	Option 1 → Mode u → securite → clé crypto	uvajkwvv
②	Déchiffrer	Option 2 → Mode u → uvajkwvv → clé crypto	securite
③	Attaque Kasiski	Option 3 → Mode t	Longueur + clé trouvées
④	Lien OTP	Option 4	Démonstration affichée
⑤	Quitter	Option 5	—

■ Vigenère : clé répétitive. Kasiski trouve la longueur par séquences répétées, puis IC retrouve chaque lettre.

1.4 — OTP / Vernam (OTP.py)

```
python Classical_Ciphers/OTP.py
```

Étape	Action	Entrées à taper	Résultat attendu
①	Chiffrer/Déchiffrer	Option 1 → Mode u → cryptographie	chiffré hex + déchiffré = cryptographie

Étape	Action	Entrées à taper	Résultat attendu
②	Two-time pad	Option 2	$C1 \oplus C2 = M1 \oplus M2$, crib dragging visible
③	Quitter	Option 3	—

■ OTP = XOR avec clé aussi longue que le message. Sécurité parfaite si clé utilisée UNE SEULE fois.

1.5 — Hill (Hill.py)

```
python Classical_Ciphers/Hill.py
```

Étape	Action	Entrées à taper	Résultat attendu
①	Chiffrer	Option 1 → Mode t	hello world chiffré, matrice $[[3,3],[2,5]]$
②	Déchiffrer	Option 2 → Mode t	retour à helloworld
③	Attaque clair connu	Option 3 → Taille 2 → paire (help / hiat)	Matrice clé $[[3, 3], [2, 5]]$ retrouvée ■
④	Quitter	Option 4	—

■ Hill : multiplication matricielle mod 26. Attaque clair connu : $K = C \cdot P^{-1}$.

1.6 — Playfair (Playfair.py)

```
python Classical_Ciphers/Playfair.py
```

Script de test automatique (pas de menu). Ce qui s'affiche :

Message	Clé	Chiffré	Déchiffré
hello world	security	FUOQMPXNSPHQ	HELXLOWORLDX

■ Playfair travaille sur des bigrammes dans une grille 5×5. J et I sont fusionnés.

1.6 bis — Hill et Playfair combinés (Hill_et_playfair.py)

```
python Classical_Ciphers/Hill_et_playfair.py
```

Script qui exécute les deux chiffrements séquentiellement.

Chiffrement	Message	Clé/Matrice	Chiffré
Hill	hello world	$[[3, 3], [2, 5]]$	hiozeipjql
Playfair	hello world	security	FUOQMPXNSPHQ

■ Ce script démontre l'implémentation des deux algorithmes en utilisant NumPy pour l'inversion matricielle de Hill.

1.7 — Rail Fence (rail_fence_cipher.py)

```
python Classical_Ciphers/rail_fence_cipher.py
```

Script de test automatique (pas de menu). Ce qui s'affiche :

Message	Rails	Chiffré	Déchiffré
HELLO WORLD	3	HOREL OLLWD	HELLO WORLD

■ Rail Fence : transposition zigzag sur N rails, lecture ligne par ligne.

1.8 — Analyse Fréquentielle (AnalyseFréquentielle.py)

```
python Classical_Ciphers/AnalyseFréquentielle.py
```

Script automatique : analyse un long texte chiffré et affiche un histogramme (Observé vs Théorique).

Ce qui s'affiche	Valeur / Description
Fréquence de chaque lettre	Ex: z: 6.32%, x: 5.87%, ...

Ce qui s'affiche	Valeur / Description
Histogramme matplotlib	Barres bleues (observé) vs vertes (fréquences françaises théoriques)
Interprétation	Si la courbe est plate → chiffrement poly-alphabétique

■ L'analyse fréquentielle exploite le fait que certaines lettres (E, A, S en français) apparaissent plus souvent. Un chiffrement monoalphabétique conserve cette distribution.

1.9 — Indice de Coïncidence (*IndiceDeCoincidence.py*)

```
python Classical_Ciphers/IndiceDeCoincidence.py
```

Étape	Action	Entrées à taper	Résultat attendu
①	Test automatique	t	IC ≈ 0.074 (texte français)
②	Texte aléatoire court	u → « aaabbbccc »	IC plus élevé (répétitions)
③	Texte chiffré César	u → fubswrjudsklh	IC ≈ 0.065-0.074 (mono-alpha)

■ $IC = \frac{\sum(n_i * (n_i - 1))}{(N * (N - 1))}$. Français ~ 0.074, Anglais ~ 0.066, Aléatoire ~ 0.038. Un IC bas indique un chiffrement poly-alphabétique.

1.10 — Substitution Aléatoire (*Substitution_Aleatoire.py*)

```
python Classical_Ciphers/Substitution_Aleatoire.py
```

Script automatique : génère une table de substitution aléatoire, chiffre et déchiffre « HELLO WORLD ».

Ce qui s'affiche	Exemple
Table de substitution	{'a': 'x', 'b': 'm', ...} (26 mappages aléatoires)
Texte chiffré	Variable à chaque exécution (ex: NKZZB ABCZS)
Texte déchiffré	HELLO WORLD (retrouvé via table inverse)

■ Substitution monoalphabétique : $26! = 4 \times 10^{26}$ clés possibles. Vulnérable à l'analyse fréquentielle mais résistant à la force brute pure.

1.11 — Test de Kasiski (*TestDeKasiski.py*)

```
python Classical_Ciphers/TestDeKasiski.py
```

Étape	Action	Entrées à taper	Résultat attendu
①	Analyser exemple	Option 1 → t	Séquences répétées + PGCD → longueur clé trouvée
②	Quitter	Option 2	—

■ Kasiski : cherche les trigrammes répétés dans le texte chiffré. Les écarts entre répétitions sont multiples de la longueur de clé. Le PGCD donne la longueur probable.

Ordre suggere pour l'enregistrement

#	Fichier	Durée est.
1	Classical_Ciphers/Cesar.py	3 min
2	Classical_Ciphers/affine.py	2 min
3	Classical_Ciphers/vigenere.py	2 min
4	Classical_Ciphers/OTP.py	2 min
5	Classical_Ciphers/Hill.py	2 min

#	Fichier	Durée est.
6	Classical_Ciphers/Playfair.py	1 min
7	Classical_Ciphers/Hill_et_playfair.py	1.5 min
8	Classical_Ciphers/rail_fence_cipher.py	1 min
9	Classical_Ciphers/AnalyseFrequentielle.py	1 min
10	Classical_Ciphers/IndiceDeCoincidence.py	1 min
11	Classical_Ciphers/Substitution_Aleatoire.py	1 min
12	Classical_Ciphers/TestDeKasiski.py	2 min

Phrases cles a prononcer

Script	Phrase clé à dire à l'oral
César	Decalage de k positions, force brute a 26 essais
Affine	Chiffrement mathématique $y = (a \cdot x + b) \bmod 26$
Vigenere	Cle repetitive, Kasiski trouve la longueur de cle
OTP	Securite parfaite si la cle est utilisee une seule fois
Hill	Multiplication matricielle mod 26, attaque clair connu
Playfair	Chiffrement par paires de lettres sur une grille 5x5
Hill+Playfair	Démonstration des deux chiffrements avec numpy
Rail Fence	Transposition en zigzag sur N rails
Analyse Freq.	Distribution des lettres trahit le chiffrement monoalphabetique
IC	IC proche de 0.074 = francais, proche de 0.038 = aleatoire
Substitution	26! cles possibles mais vulnerable a l'analyse frequentielle
Kasiski	PGCD des ecarts entre trigrammes repetes = longueur de cle

■ Laisse chaque résultat s'afficher complètement avant de continuer.